

ארכיטקטורת מערכת לוגים משולבת

Console & Centralized Shared Redis Handler

מסמך אפיון הנדסי ומימוש בקוד (Python) לניהול אחיד של לקוח Redis מבוזר

מערכת פיתוח ארכיטקטורה

יוצר:

סופי / מאושר

סטטוס:

יוני 2026

תאריך
עדכון:

פרויקט מקשר Backend-Frontend

יעד:

1. מבוא וסקירה כללית

במערכות תוכנה מודרניות, בפרט כאלו המקשרות בין צד השרת (Backend) לצד הלקוח (Frontend) בזמן אמת, קיים צורך בניהול לוגים היברידי. מצד אחד, נדרש מעקב מלא ומיידי בקונסולה לצורכי דיבוג, ניטור שוטף וזיהוי תקלות בזמן אמת. מצד שני, חלק מסוים מהלוגים (כגון אירועים קריטיים, שינויי סטטוס או עדכוני תשתית) חייב להידחף לתוך תור או רכיב תקשורת מרכזי דוגמת Redis, על מנת להעבירו לעיבוד או להצגה ישירה בפרונטאנד.

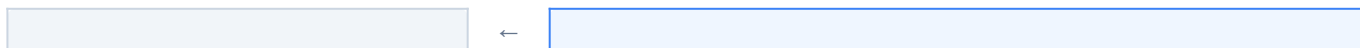
מסמך זה מגדיר ארכיטקטורה נכונה המבוססת על **עלזרקת תלויות (Dependency Injection)** במקום לייצר חיבורים כפולים או עצמאיים עבור רכיב הלוגר, המערכת עושה שימוש חוזר בקליינט ה-Redis המרכזי והקיים של הפרויקט. גישה זו מונעת עומס חיבורים (Connection Pool Saturation) ומונעת בעיות של יבוא מעגלי (Circular Imports) בקוד.

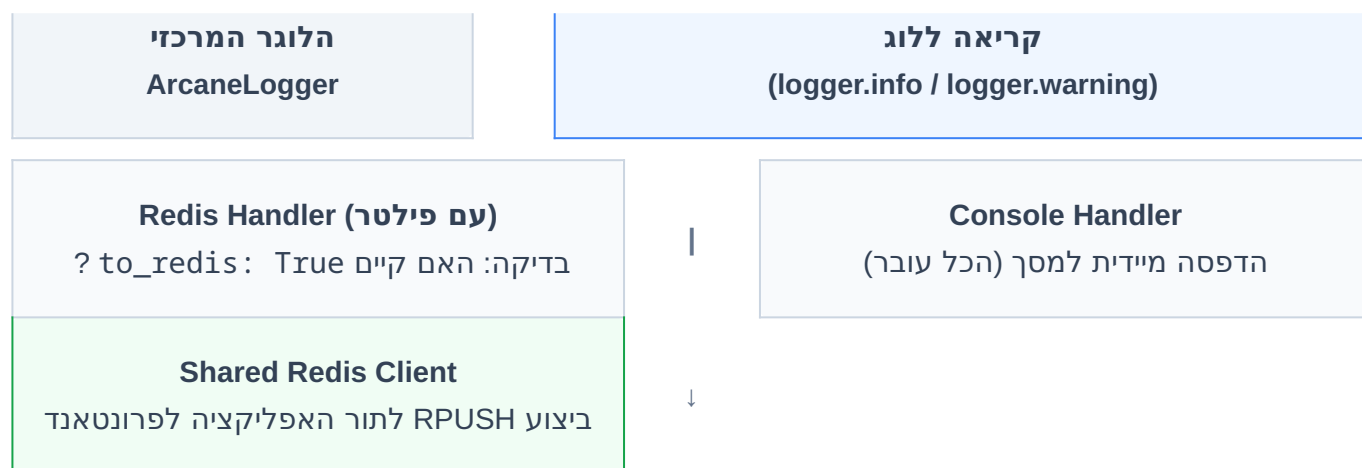
עקרון הפעולה המרכזי:

- Console Handler:** תופס ומדפיס תמיד את כל סוגי הלוגים (DEBUG ומעלה) ללא סינון.
- Redis Handler:** תופס לוגים באופן סלקטיבי בלבד, על בסיס מנגנון `logging.Filter` ייעודי הבודק קיום של דגל מפורש (`.to_redis=True`).

2. זרימת הנתונים במערכת

להלן תרשים מבני המציג כיצד הודעת לוג זורמת דרך הרכיבים השונים מרגע יצירתה ועד להפצתה ליעדים המתאימים:





3. מימוש הלוגר (logger_setup.py)

קובץ זה אחראי על הגדרת הפילטר הסלקטיבי, ה-Handler הייעודי שמדבר עם הלקוח המרכזי, ופונקציית האתחול החד-פעמית שמקבלת את הקליינט ומדווחת על סטטוס החיבור ישירות לקונסולה.

```

import logging
from redis import Redis

class RedisFilter(logging.Filter):
    """
    פילטר הבודק האם הלוג סומן במפורש לשליחה לרדיס.
    """
    def filter(self, record):
        return getattr(record, 'to_redis', False)

class RedisHandler(logging.Handler):
    """
    הנדלר מותאם אישית הדוחף הודעות לוג מפורמטות לתוך רשימת Redis.
    עושה שימוש בקליינט משותף שהוזרק מבחוץ.
    """
    def __init__(self, redis_client: Redis, list_key: str = 'app_logs'):
        super().__init__()
        self.redis_client = redis_client
        self.list_key = list_key

    def emit(self, record):
        try:
            log_entry = self.format(record)
            # דחיפת הלוג לסוף הרשימה ברדיס (מתאים לקריאה ע"י הפרונטאנד)
            self.redis_client.rpush(self.list_key, log_entry)
        except Exception:
            self.handleError(record)

def init_project_logger(redis_client: Redis = None):
    """
    פונקציית אתחול מרכזית המגדירה את הנדלר הקונסולה והרדיס.
    """
    logger = logging.getLogger('ArcaneLogger')
    logger.setLevel(logging.DEBUG)

    # ניקוי הנדלרים קיימים למניעת כפילויות במקרה של אתחול כפול
    if logger.hasHandlers():
        logger.handlers.clear()

    # הגדרת פורמט אחד ומקצועי להודעות
    formatter = logging.Formatter('%(asctime)s | %(levelname)s | %(message)s')

    # 1. הגדרת קונסולה (תמיד פעילה, מקבלת הכל)
    console_handler = logging.StreamHandler()
    console_handler.setLevel(logging.DEBUG)
    console_handler.setFormatter(formatter)

```

```

logger.addHandler(console_handler)

# 2. הזרקה וחיבור של קליינט הרדיס הקיים
if redis_client:
    try:
        # וידאו פעילות קליינט הרדיס המשותף
        redis_client.ping()
        logger.info("Shared Redis client attached to ArcaneLogger successfully.")

        redis_handler = RedisHandler(redis_client)
        redis_handler.setLevel(logging.DEBUG)
        redis_handler.setFormatter(formatter)

        # הצמדת הפילטר הסלקטיבי להנדלר של רדיס
        redis_handler.addFilter(RedisFilter())
        logger.addHandler(redis_handler)

    except Exception as e:
        logger.error(f"Failed to attach shared Redis client. Fallback to Console only. Error: {e}")
    else:
        logger.warning("Logger initialized without a Redis client. Logs will only go to Console.")

return logger

```

4. קובץ האתחול הראשי וניהול הלקוח (main.py)

בנקודת הכניסה של האפליקציה, אנו מייצרים/מייבאים את לקוח ה-Redis העצמאי שלנו שמשמש את שאר רכיבי המערכת, ומעבירים אותו כפרמטר לפונקציית האתחול של הלוגר.

```
# דוגמה לייבוא או יצירה של קליינט רדיס מרכזי ועצמאי
import redis
from logger_setup import init_project_logger

# יצירת החיבור הראשי של הפרויקט (משרת את כל הלוגיקה, לא רק לוגים)
my_shared_redis_client = redis.Redis(
    host='localhost',
    port=6379,
    db=0,
    decode_responses=True
)

# אתחול הלוגר והזרקת הקליינט.
# בשלב זה יודפס לקונסולה סטטוס החיבור לרדיס!
logger = init_project_logger(redis_client=my_shared_redis_client)

logger.info("The application initialization sequence has started.")
logger.warning("Example of global issue: API payload structure modified.",
    extra={'to_redis': True})
```

5. שימוש שוטף ברחבי האפליקציה (modules)

בכל קובץ, מודול או פונקציה אחרת בפרויקט, אין צורך לייבא את קליינט הרדיס או את פונקציית האתחול מחדש. קוראים ללוגר לפי שמו השמור בזיכרון של פייתון (ArcaneLogger) ומשתמשים בו בצורה פשוטה:

```
import logging

# משיכת הלוגר הקיים והמקונפג מתוך הזיכרון
logger = logging.getLogger('ArcaneLogger')

def process_user_action(user_id, action_type):
    # לוג רגיל - יודפס רק בקונסולה לצורכי מעקב פנימי
    logger.debug(f"User {user_id} performed temporary internal action.")

    if action_type == "critical_payment":
        # לוג שיופס בקונסולה *וגם* יידחף ישירות לרדיס לטובת הפרוטטאנד
        logger.info(
            f"User {user_id} completed a critical payment transaction.",
            extra={'to_redis': True}
        )
```

6. דגשים ארכיטקטוניים ויתרונות השיטה

- **מניעת נעילת חיבורים (Connection Overhead):** מכיוון שלא נפתח לקוח Redis חדש לצורך ה-Logger, המערכת שומרת על ה-Connection Pool המקורי מבלי לייצר הגדלה מיותרת של כמות החיבורים מול שרת ה-Redis.
- **בידוד מוחלט מיבוא מעגלי (Circular Imports):** רכיבי המערכת השונים לא צריכים לייבא אחד את השני לצורך הדפסת לוגים. הקריאה ל-`logging.getLogger('ArcaneLogger')` מבוססת על מנגנון הרישום הגלובלי המובנה של Python.
- **חוסן מובנה (Fault Tolerance):** אם שרת ה-Redis נופל או חווה ניתוק, מנגנון ה-`try-except` בתוך ה-`Emit` של ה-Handler ימנע מהאפליקציה לקרוס, והלוגים ימשיכו לזרום כסדרם לקונסולה המקומית.

המלצה להרחבה עתידית (Asynchronous Logging):

בסביבות ייצור (Production) בעלות עומס עבודה קיצוני, מומלץ לעטוף את ה-Handlers בעזרת `logging.handlers.QueueHandler`. הדבר מאפשר ביצוע פעולות כתיבה לקונסולה ולרדיס בצורה אסינכרונית לחלוטין על גבי תהליכון (Thread) נפרד, על מנת שלא לעכב את ה-Main Thread של הלוגיקה העסקית.